Forecasting Spanish electricity prices, and market state clustering using Machine learning techniques

MSc Artificial Intelligence

Faculty of Computing, Engineering, and the Built Environment

Module: CMP7228 Machine Learning

Student Name: Salaheddine Chouikh

Student Number: 24156735

Contents

Contents 1
Introduction to Electricity Price Forecasting and Abstarct: 5
Problem definition: 5
Datasets explanation: 6
Time series dataset 6
ENERGY_DATASET.csv: 6
WEATHER_DATASET.csv: 6
Datasets Preprocessing and EDA: 7
Preprocessing: 7
ENERGY_DATASET.csv 7
Data types: 7
Dealing with invalid values: 8
After dropping the columns: 8
Trying to detect when the values are missing: 9
Visualising the values to get a better understanding of the situation: 9
Filling null values using interpolation: 10
Weather_features.csv: 10
Let's check the columns in our dataset: 11
Dropping temperature columns: 11
Checking Weather column: 12
Checking the data after the dropping of columns 13
Checking how many unique values are in the city_name column: 13
Splitting and merging the datasets: 16
EDA and visual analysis: 16
Seasonality and trend analysis: 17
Conclusion: 19
Bivariate EDA: 20
ML Problem 1(Unsupervised): Feature engineering using dimensionality reduction. 20
Execution time comparison: 21
Reducing all similar features using PCA: 22
Cumulative explained Variance Ratio: 22
Creating new dataset with reduced features: 23
ML Problem 2 (Supervised): Forecasting electricity prices. 24
Characteristics of our dataset: 24

Splitting the dataset to 80% training and 20% testing: 24
The selected models: 24
Prophet: (made by Facebook) 24
Benchmarking other models: 25
The metrics used to evaluate our models: 25
Croston: 26
Prophet: 26
These metrics are for the final_dataset before and after dimensionality reduction (reduced_dataset) 26
Tunning our prophet model: 27
Metrics conclusion: 27
Forecast of the testing split of the dataset: 27
Croston: 27
Prophet with the non-feature engineered dataset: 28
Prophet with the feature engineered dataset: 28
Conclusion: 28
Predicting future electricity prices (7 days): 29
Prophet with the non-feature engineered dataset: 29
Prophet with the feature engineered dataset: 29
ML Problem 3 (Unsupervised): Clustering based on the correlated feature with the grid load and electricity price. 30
Methods and metrics used to get the best k-number: 31
Elbow method: 31
average silhouette score: 31
Conclusion: 30
The results of clustering using k-means: 32
Raw results: 32
Plotting the results: 33
Conclusion: 33
References: 34

Dataset Kaggle link	https://www.kaggle.com/datasets/nicholasjhana/energy-consumption-generation-prices-and-weather
Energy_dataset.csv preprocessing colab file	https://colab.research.google.com/drive/1izOffLcdJz2b6s4IpSE7Ay-a2DOznCiV?usp=sharing
EDA and Weather_features.csv preprocessing colab file	https://colab.research.google.com/drive/1QaYJs6n3hwZwpbV4D0ZRUBlxRwb1EPfh?usp=sharing
Problem1 colab file	https://colab.research.google.com/drive/1VovwWKyZFr4iwQuxm28OREqm5eqmMchp?usp=sharing
Problem 2 colab file with pre-processed dataset	https://colab.research.google.com/drive/1PmhEORltAx1XPpwyntWavmKzgK_nLTL7?usp=sharing
Problem 2 colab file with pre-processed reduced dataset	https://colab.research.google.com/drive/1fB3C6-4sA2sWg_2LfVbdmaSkVTV5BdhQ?usp=sharing
Problem3 colab file	https://colab.research.google.com/drive/1iwFCpyz9VI3KYnZjO40WcuBAhHhmomst?usp=sharing

Introduction to Electricity Price Forecasting and Abstract:

Electricity price forecasting plays a crucial role in energy economics and power systems management. By accurately predicting future electricity prices, stakeholders can optimize their operations, reduce risks, and make more informed decisions. The forecasting process typically relies on historical data, including grid load, weather conditions, seasonality trends, and energy generation sources. As renewable energy sources such as solar and wind become more integrated into the grid, forecasting has become more complex because of how inconsistent weather is, but also more critical for ensuring grid stability and economic sustainability (Weron, 2014).

With the increasing volatility of energy prices and the need for greater grid efficiency, reliable price forecasting helps market participants plan effectively. It supports the integration of renewable energy into the power grid by optimizing resource allocation, ensuring economic stability, and contributing to sustainability efforts (Nowotarski & Weron, 2018).

This report aims to develop and compare two different supervised machine learning models for predicting electricity prices in the Spanish electricity market. These models will incorporate a variety of features, some of which will be engineered using unsupervised machine learning techniques. Key features include weather data, energy generation statistics, and market trends, all contributing to hourly price predictions. Additionally, we will apply unsupervised learning to cluster market states, providing insights into market dynamics. By evaluating the effectiveness of these models, the report seeks to deliver more accurate electricity price forecasts while also offering valuable clustering insights that can inform better decision-making for energy producers, consumers, and policymakers.

Problem definition:

Electricity price forecasting is a challenging task due to market randomness and the influence of multiple factors such as weather conditions, energy demand, and generation sources. This project focuses on the Spanish electricity market, utilizing hourly data from 2015–2019, including:

- **Weather data**: Hourly conditions (e.g., temperature, wind speed, rainfall) from five major cities representing diverse geographic and demographic coverage.
- **Energy data**: Hourly electricity generation by various sources (e.g., fossil gas, coal, wind), total grid load, and electricity prices per Megawatt.

The goal is to develop a machine learning model that accurately predicts electricity prices by integrating these datasets and can identify different market states all for better load management.

Datasets explanation:

Time series dataset

Our data is set in a time series dataset, which means every row is a snapshot of the continuous data. “Data points are arranged in a sequence based on time, allowing for the analysis of trends and patterns over time” (Prasad, 2024).

In our case the frequency of our dataset is by the Hour, from start of 2015 till the end of 2018, so 4 years’ worth of data. “Observations are taken at consistent intervals, which is vital for maintaining the integrity of the analysis” (Le, 2024)

ENERGY_DATASET.csv:

The dataset contains hourly energy-related information, including:

Clean energy generation (solar, wind, hydraulic) by MW/h.

Unclean energy generation (biomass, fossil fuels...)

Total load carried by the grid in Spain in MW/h, actual and forecasted.

Electricity prices (day-ahead and actual) in MW/Euro.

WEATHER_DATASET.csv:

This dataset provides hourly weather data for five major cities in Spain: Madrid, Barcelona, Valencia, Seville, and Bilbao. Each city represents a key geographic region, ensuring broad coverage of Spain's territory. The dataset includes the following weather features:

Temperature: Hourly temperature in Kelvin.

Wind Speed: Hourly average wind speed in meters per second.

Humidity: Hourly relative humidity as a percentage.

Rainfall: Hourly rainfall in millimetres.

Qualitative Weather Descriptions: Text-based and numerical descriptions of weather conditions, providing additional context for each hour.

This dataset also has a column for **city_name,** the total of cities is 5 cities, which are scattered around Spain, and their population is a third of all of Spain, which mean the dataset is giving a whole representative data for all of Spain.

Datasets Preprocessing and EDA:

Preprocessing:

Goals:

- ✓ Understand the structure of the dataset.
- ✓ Identify missing values, anomalies, or outliers.
- ✓ Visualize relationships between features.

Some applications utilized for pre-processing were inspired from a Kaggle notebook, which is appropriately referenced below (Roussis, 2023).

ENERGY_DATASET.csv:

Colab file is in the table page 4.

Data types:

As we can see the column time is not in the correct format, so we need to convert it to a date format, and then we can use it as an index for our dataset.

The other columns seem in the right Dtype.

#	Column	Non-Null Count	Dtype
0	time	35064 non-null	object
1	generation biomass	35045 non-null	float64
2	generation fossil brown coal/lignite	35046 non-null	float64
3	generation fossil coal-derived gas	35046 non-null	float64
4	generation fossil gas	35046 non-null	float64
5	generation fossil hard coal	35046 non-null	float64
6	generation fossil oil	35045 non-null	float64
7	generation fossil oil shale	35046 non-null	float64
8	generation fossil peat	35046 non-null	float64
9	generation geothermal	35046 non-null	float64
10	generation hydro pumped storage aggregated	0 non-null	float64
11	generation hydro pumped storage consumption	35045 non-null	float64
12	generation hydro run-of-river and poundage	35045 non-null	float64
13	generation hydro water reservoir	35046 non-null	float64
14	generation marine	35045 non-null	float64
15	generation nuclear	35047 non-null	float64
16	generation other	35046 non-null	float64
17	generation other renewable	35046 non-null	float64
18	generation solar	35046 non-null	float64
19	generation waste	35045 non-null	float64
20	generation wind offshore	35046 non-null	float64
21	generation wind onshore	35046 non-null	float64
22	forecast solar day ahead	35064 non-null	float64
23	forecast wind offshore eday ahead	0 non-null	float64
24	forecast wind onshore day ahead	35064 non-null	float64
25	total load forecast	35064 non-null	float64
26	total load actual	35028 non-null	float64
27	price day ahead	35064 non-null	float64
28	price actual	35064 non-null	float64

Figure 1: columns data types in energy_dataset.

Dealing with invalid values:

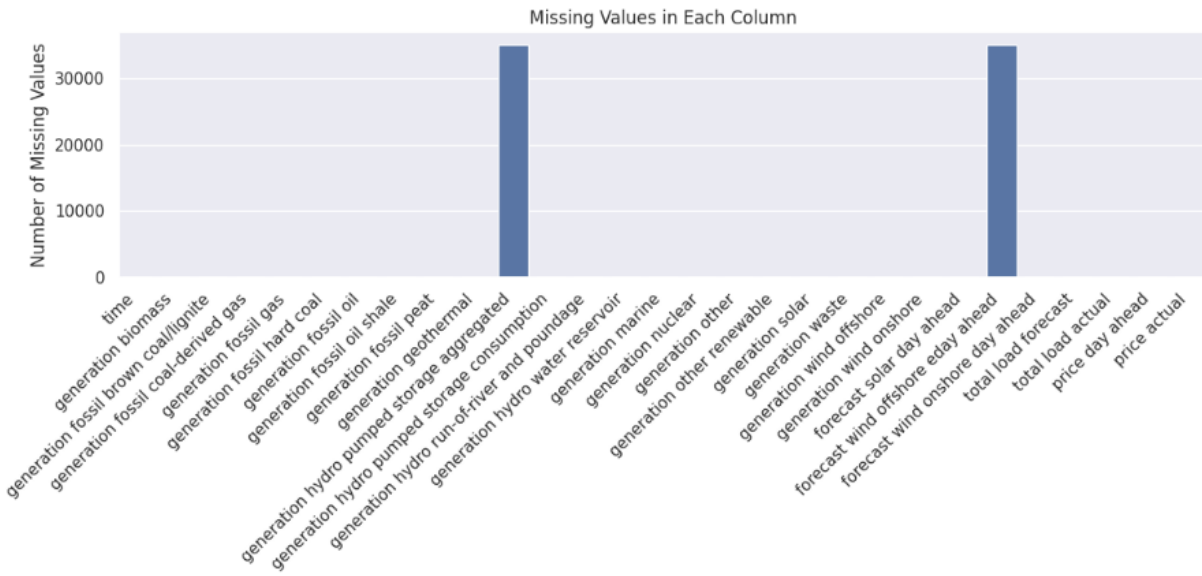

Figure 2: Missing values in the energy_dataset.

Two columns do not have any valid values at all, so we will just drop them with some other non-related columns to our study.

After dropping the columns:

Now we will see the if there are any invalid values in our dataset. On the good side, **price actual** has no NaNs, which will be our target for the model.

Total load actual is the column with the greatest number of missing values.

The missing values for other columns are close, which could mean that they are in the same data points.

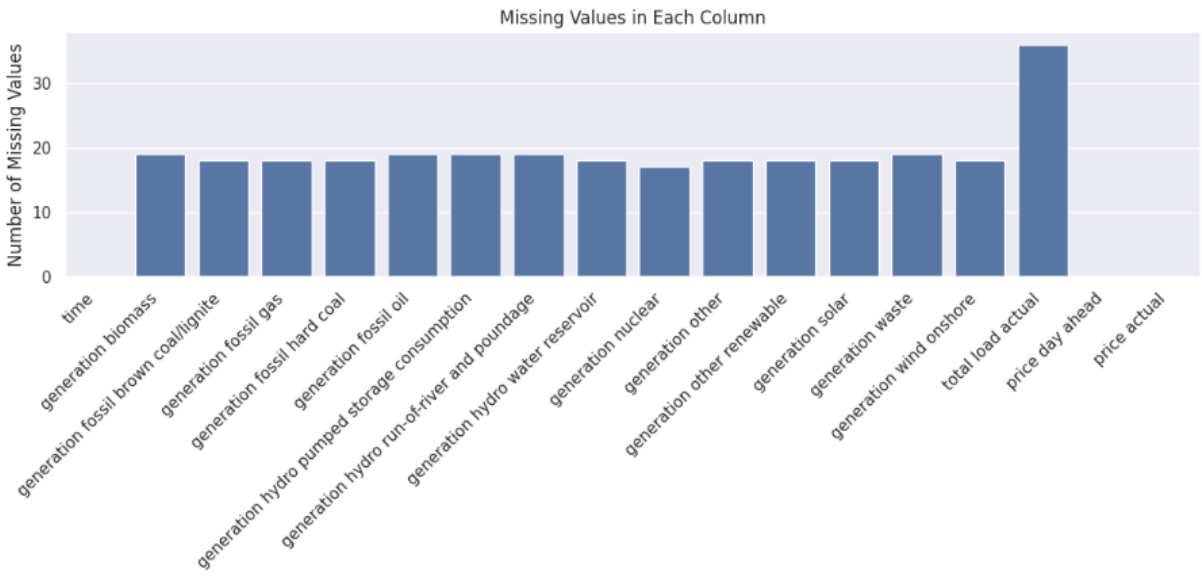

Figure 3: Missing values in the energy_dataset.

Energy dataset has no duplicated values. However, it has many null values, and as this is a time series we cannot simply just drop the rows, a better solution would be to impute the values.

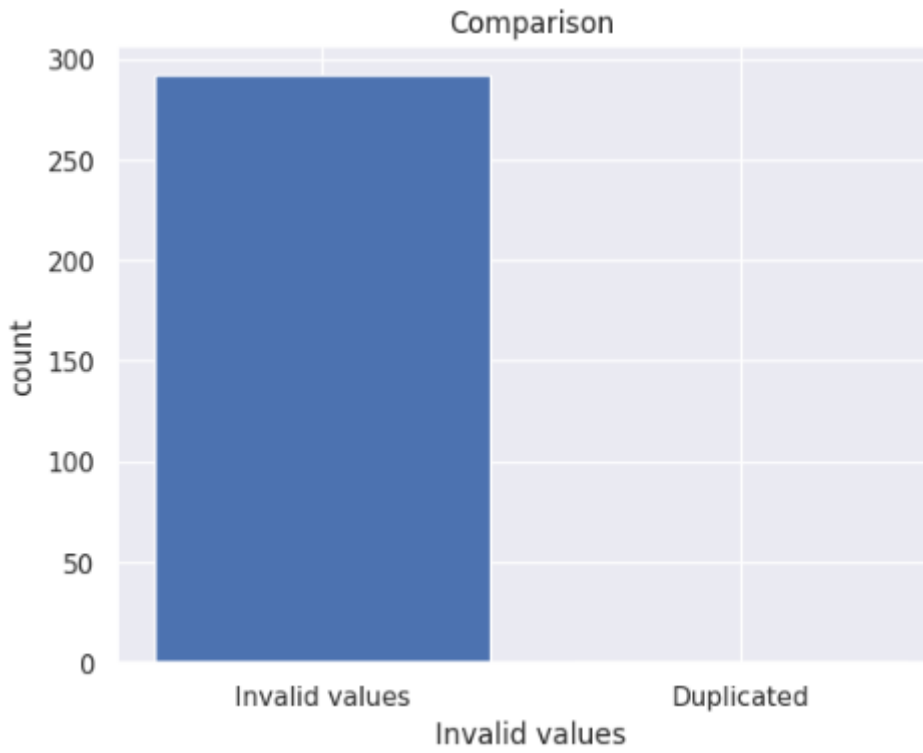

Figure 4: Invalid values vs Duplicates.

Trying to detect when the values are missing:

Some values are missing in the second week of our dataset, we will focus on the column **total load actual** to get better insights, because it has the biggest number of NaN values.

time	generation biomass	generation fossil brown coal/lignite	generation fossil gas	generation fossil hard coal	generation fossil oil	hydro pumped storage consumption	hydro run-of-river and poundage	generation hydro water reservoir	generation nuclear	generation other	generation other renewable	generation solar	generation waste	generation wind onshore	total load actual
2015-01-05 02:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	21182.0
2015-01-05 11:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2015-01-05 12:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2015-01-05 13:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2015-01-05 14:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2015-01-05 15:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2015-01-05 16:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2015-01-19 18:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	39304.0
2015-01-19 19:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	39262.0

Figure 5: inspecting weather_features datapoints.

Visualising the values to get a better understanding of the situation:

We can see the missing values in the plot, since our data is a time series and is continuous, we can impute the data using **interpolation**. Interpolation is used to create

continuous paths from discrete time-series inputs, improving the accuracy of predictions in machine learning tasks (Jhin et al., 2022).

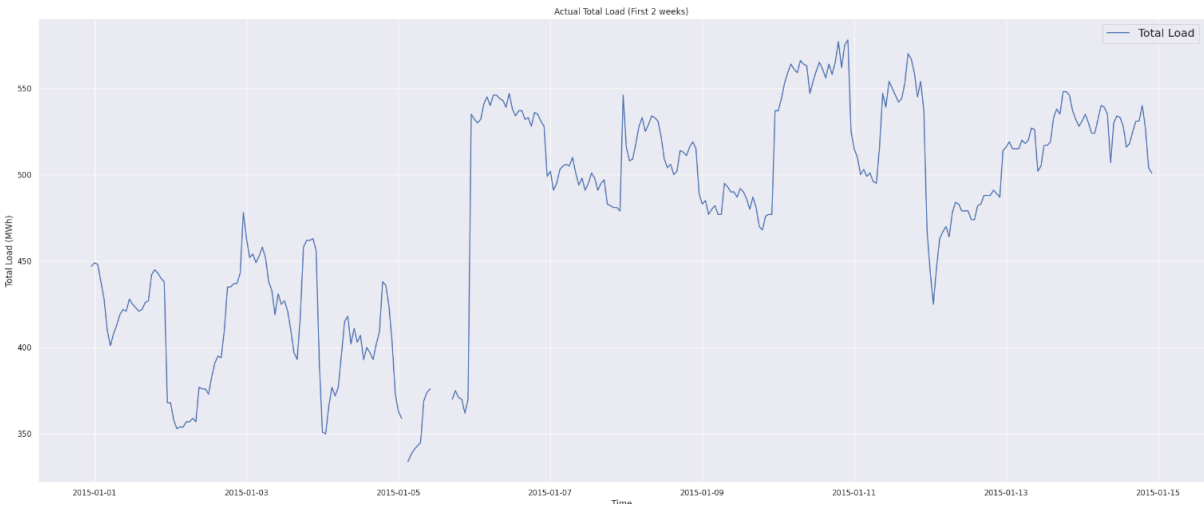

Figure 6: Plot of the actual load column showing missing values.

Filling null values using interpolation:

Our data have been successfully imputed and filled.

Figure 7: Plot of the actual load column imputed.

Weather_features.csv:

Colab file is in the table page 4.

the total number of rows in this dataset is 170k data points, and this is 5 times more than **energy_dataset.csv,** the reason its more is because the 5 cities do not have separated columns, they are all in the same column named **city_name**, which means that the dataset gives **weather features** for each city once in a time, making the dataset 5 times bigger than we had columns for each city and its features, and this is what we are trying to achieve in the

preprocessing.
Achieving that will help with merging our two datasets.

Let us check the columns in our dataset:

we can see some columns that are not important or give the same data in diverse ways.

- we have the column temp, temp_min, temp_max all of them giving almost the same data always.

	temp	temp_min	temp_max
count	65989.00	65988.00	65988.00
mean	289.26	288.39	290.08
std	8.39	8.33	8.68
min	264.13	264.13	264.13
25%	283.11	282.15	283.33
50%	288.70	288.15	289.15
75%	295.40	294.24	296.56
max	313.33	312.15	316.48

Figure 8: description of temperature columns.

- there are four columns about weather that have same indication, talking about different states of the weather.

weather_id	weather_main	weather_description	weather_icon
803.0	clouds	broken clouds	04n
803.0	clouds	broken clouds	04n
800.0	clear	sky is clear	01d
800.0	clear	sky is clear	01d
NaN	NaN	NaN	NaN

Figure 9: weather columns.

Dropping temperature columns:

The temp column probably gives the value of the temperature by the end of each hour, and not a mean value, either way, we can use a single column of the three since they almost indicate the same thing, so we will leave only temp column, this method improves the retrieval of functionally similar molecules by focusing on essential features while discarding irrelevant ones (Nasser et al., 2022).

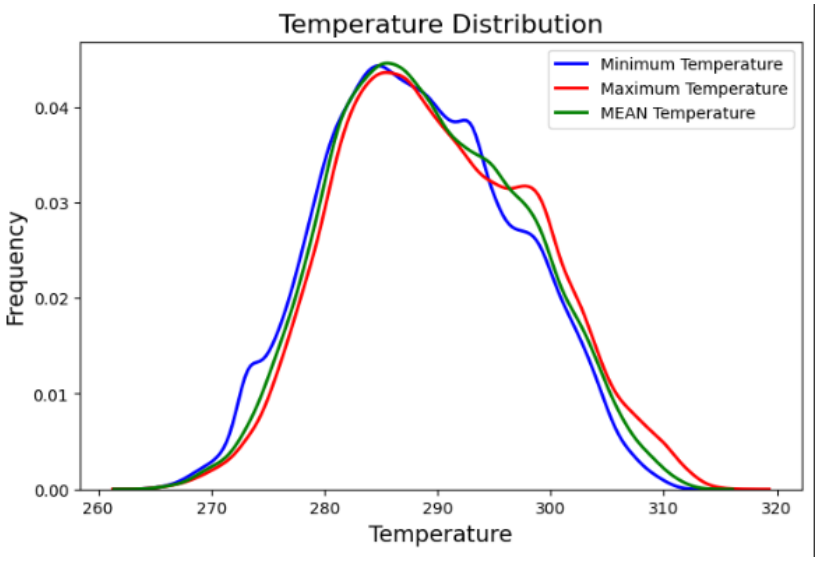

Figure 10: line graph of temperature distribution.

Checking Weather column:

The weather columns have indication about the weather if it is clear or cloudy, all of them, which brings us to the same point, but the difference in this case is that the columns are different in the amount of 'information' they give.

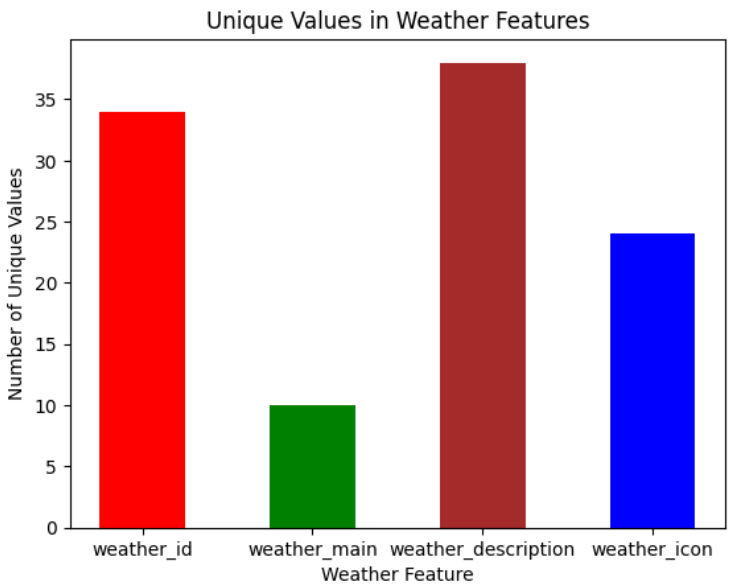

Figure 11: Unique values in weather features.

Weather_main has the least amount of 'details', while weather_id and weather_description have the most, meaning that we will leave one of them, and drop the rest. we left the weather_description as it has the greatest number of unique values.

Checking the data after the dropping of columns

This is how our data is, the string columns being the city names, weather description and the time column.

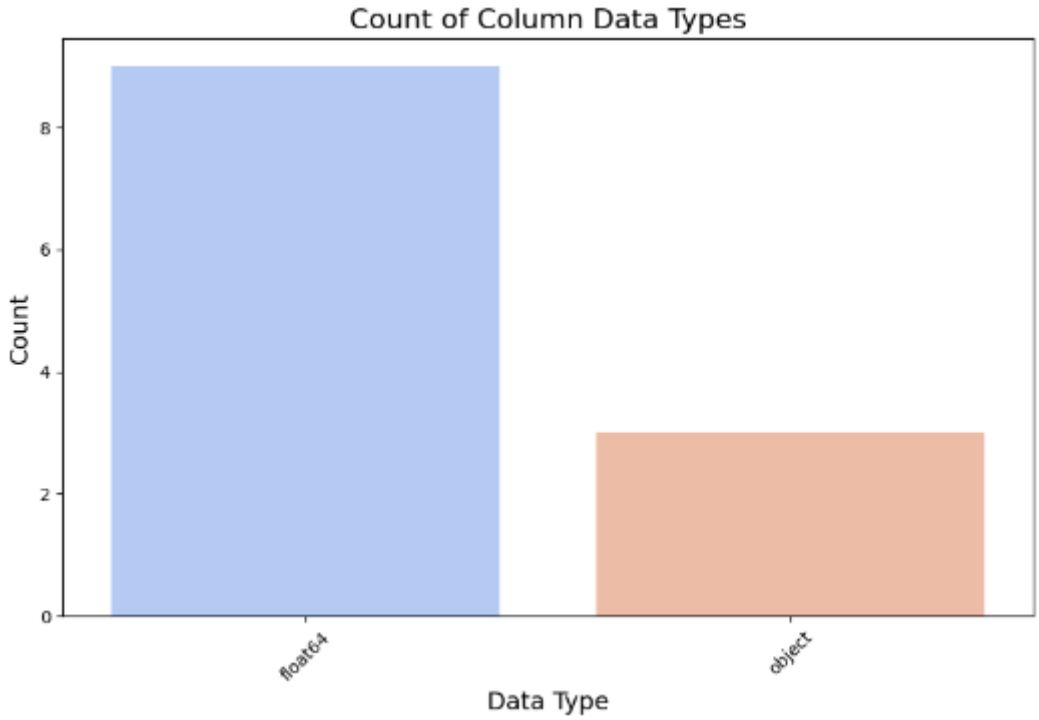

Figure 12:data types of weather_description.

Checking how many unique values are in the city_name column:

In our case, this dataset has weather of five cities in Spain, so the city_name column has 5 unique values:

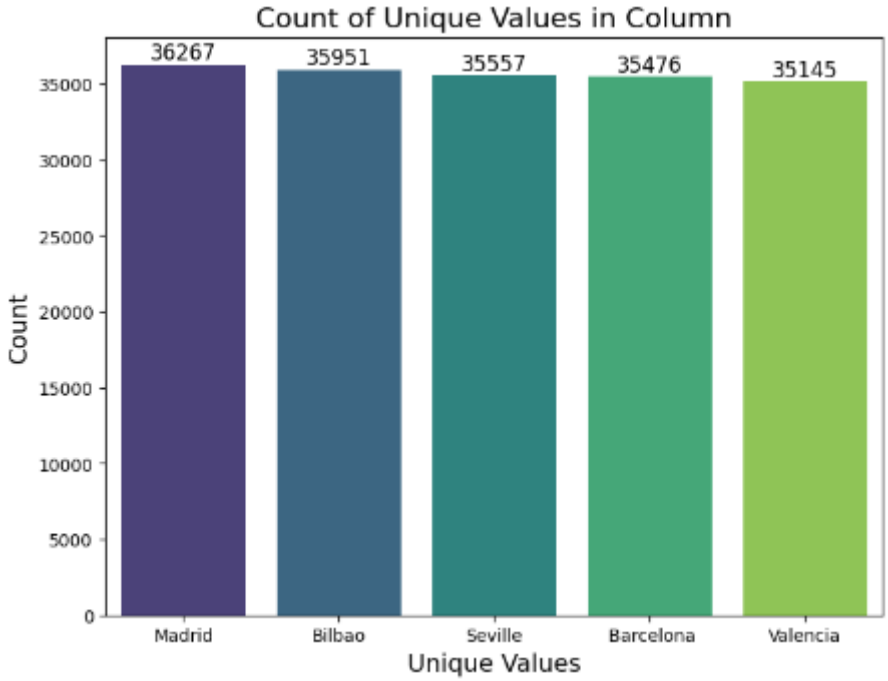

Figure 13: number of unique data point with each city name.

The cities are not equally distributed in the dataset, but each one has more than the rows of the energy_dataset, meaning that we can see after cleaning they should all have the same count of 35064.

We should be careful not to drop any unique time stamp, because this is a time series, but we are sure that there are no missing values, thanks to dataset.**describe():**

	temp	temp_min	temp_max	pressure	humidity	wind_speed	wind_deg	rain_1h	rain_3h	snow_3h	clouds_all	weather_id
count	178396.00	178396.00	178396.00	178396.00	178396.00	178396.00	178396.00	178396.00	178396.00	178396.00	178396.00	178396.00

Figure 14: count of all columns in weather_features.csv.

This means that the extra values are duplicates.

After dropping the duplicated datapoints we can see the following:

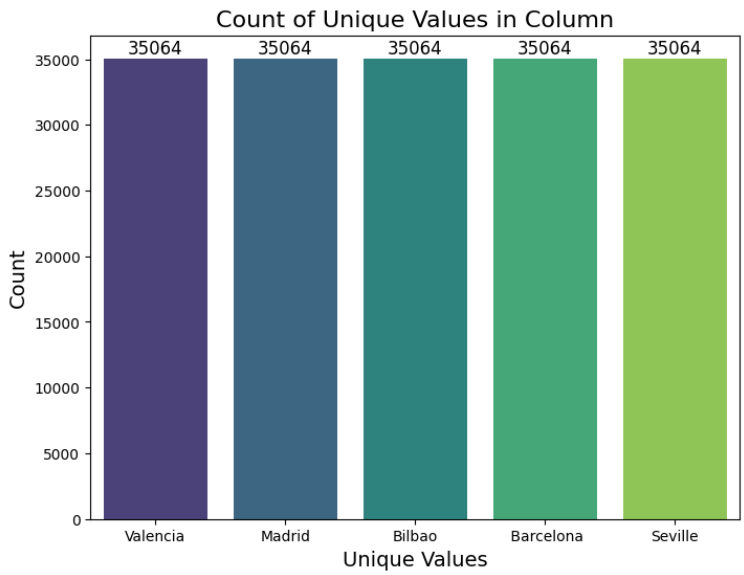

Figure 15: cleaned city_name.

Now the cities have the same data point as **energy_dataset.csv**, meaning we are ready to split our **weather_feature.csv** dataset and merge everything, but before that, let us fix some abnormal values in the dataset.

Checking abnormal values in the dataset:

Pressure:

The max value of pressure is far more (one hundred times) than 75% of the dataset.

	temp	temp_min	temp_max	pressure	humidity	wind_speed	wind_deg	rain_1h	rain_3h	snow_3h	clouds_all	weather_id
count	178396.00	178396.00	178396.00	178396.00	178396.00	178396.00	178396.00	178396.00	178396.00	178396.00	178396.00	178396.00
mean	289.62	288.33	291.09	1069.26	68.42	2.47	166.59	0.08	0.00	0.00	25.07	759.83
std	8.03	7.96	8.61	5969.63	21.90	2.10	116.61	0.40	0.01	0.22	30.77	108.73
min	262.24	262.24	262.24	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	200.00
25%	283.67	282.48	284.65	1013.00	53.00	1.00	55.00	0.00	0.00	0.00	0.00	800.00
50%	289.15	288.15	290.15	1018.00	72.00	2.00	177.00	0.00	0.00	0.00	20.00	800.00
75%	295.15	293.73	297.15	1022.00	87.00	4.00	270.00	0.00	0.00	0.00	40.00	801.00
max	315.60	315.15	321.15	1008371.00	100.00	133.00	360.00	12.00	2.32	21.50	100.00	804.00

Figure 16: description of weather_feature dataset.

Plotting the dataset will help understand the situation:

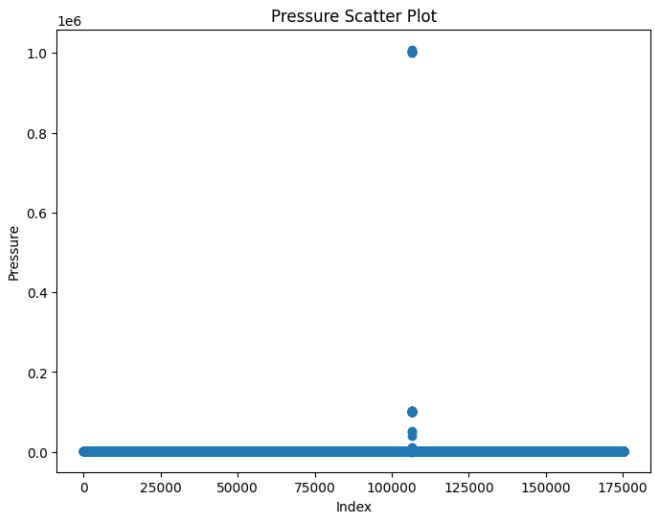

Figure 17: scatter plot of pressure column in the whole dataset.

We can see around four abnormal values, but instead of targeting those few points, we can set a rule to transform any abnormal pressure value ever recorded worldwide (more than 1,085, and less than 870 according to Wikipedia) to a NaN value, then fill them using **interpolation**.

This is the zoomed-in (datapoints 108000 to 110000) plot before and after using interpolate.

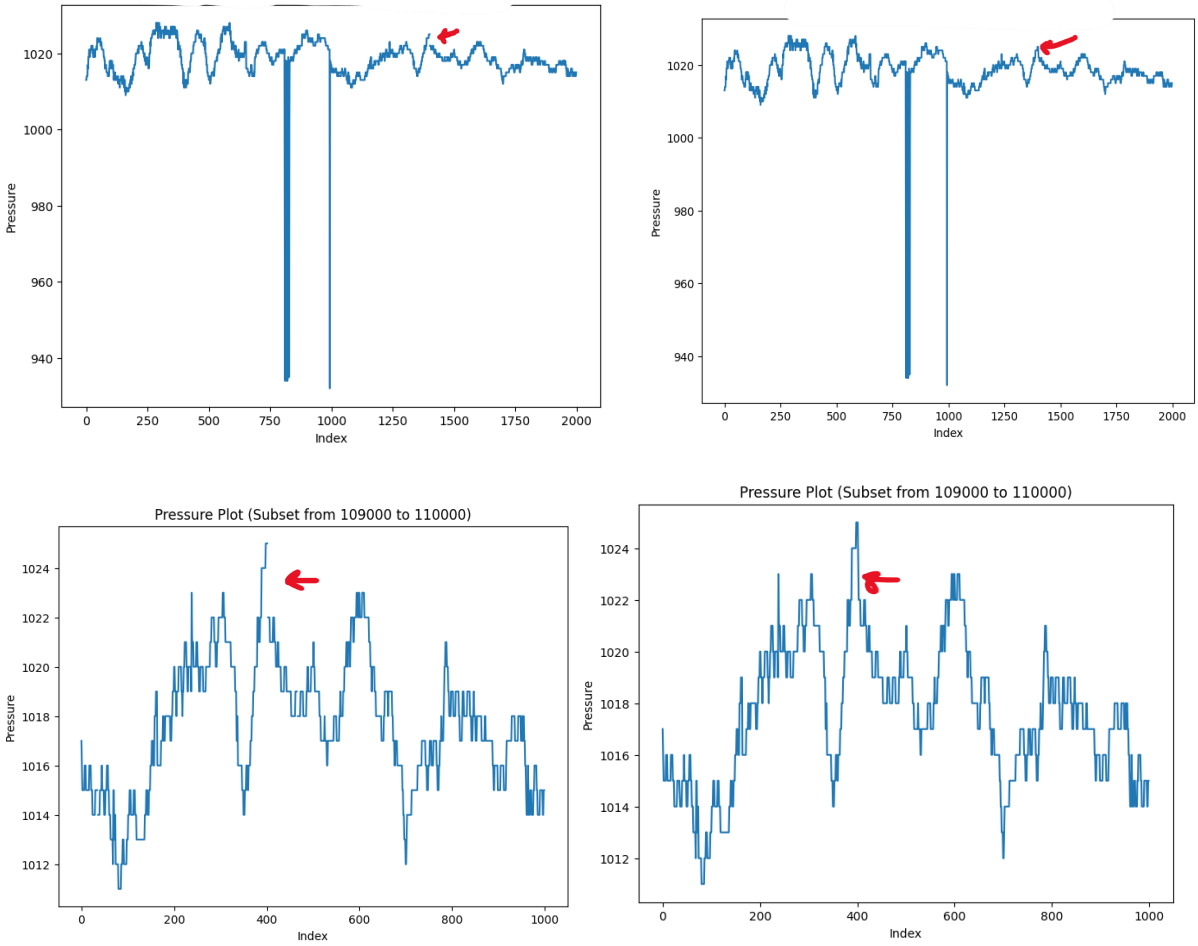

Figure 18: the interpolation difference.

Splitting and merging the datasets:

Let us split the dataset into 5 datasets for each city to investigate more.

```
Madrid    (35064, 10)
Bilbao    (35064, 10)
Seville   (35064, 10)
Barcelona (35064, 10)
Valencia  (35064, 10)
```

Figure 19: five weather_features datasets for each city name.

And then we merge all the datasets, into one big dataset with **energy_dataset.csv**.

```
(35064, 63)
```

Figure 20: shape of the final dataset.

EDA and visual analysis:

Colab file is in the table page 4.

The exploratory data analysis (EDA) focuses primarily on the **energy_dataset.csv** section of the final dataset, as it contains the most valuable and technically insightful information.

Seasonality and trend analysis:

- **Autocorrelation with past values (datapoints):**

The data points in the dataset are highly correlated with the data points before them. The periodic peaks suggest strong seasonality in the data. This confirms that the time series has repeated patterns. This behaviour extends up to 40 hours for each data point, as evident from the consistent peaks. Additionally, the increasing correlation in the later lags suggests that the seasonality continues beyond 40 hours, indicating longer-term dependencies (Proietti & Pedregal, 2022; Hyndman & Athanasopoulos, 2018).

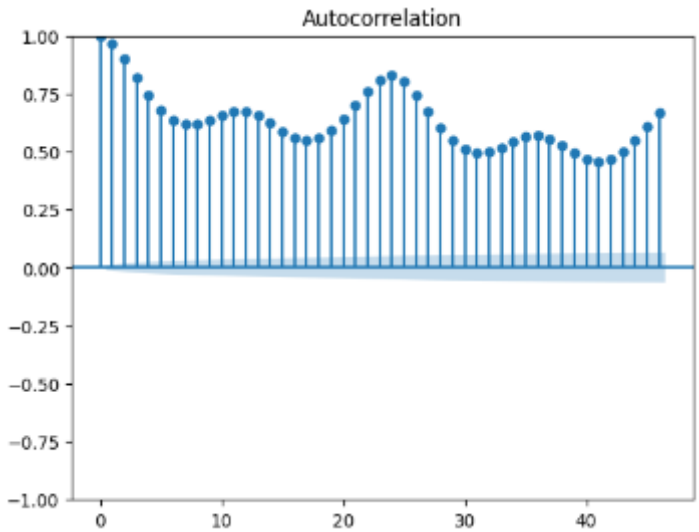

Figure 21: the datasets data points correlation with values from the past (data points before it).

- **Daily Load demand seasonality (24 Hours)**:

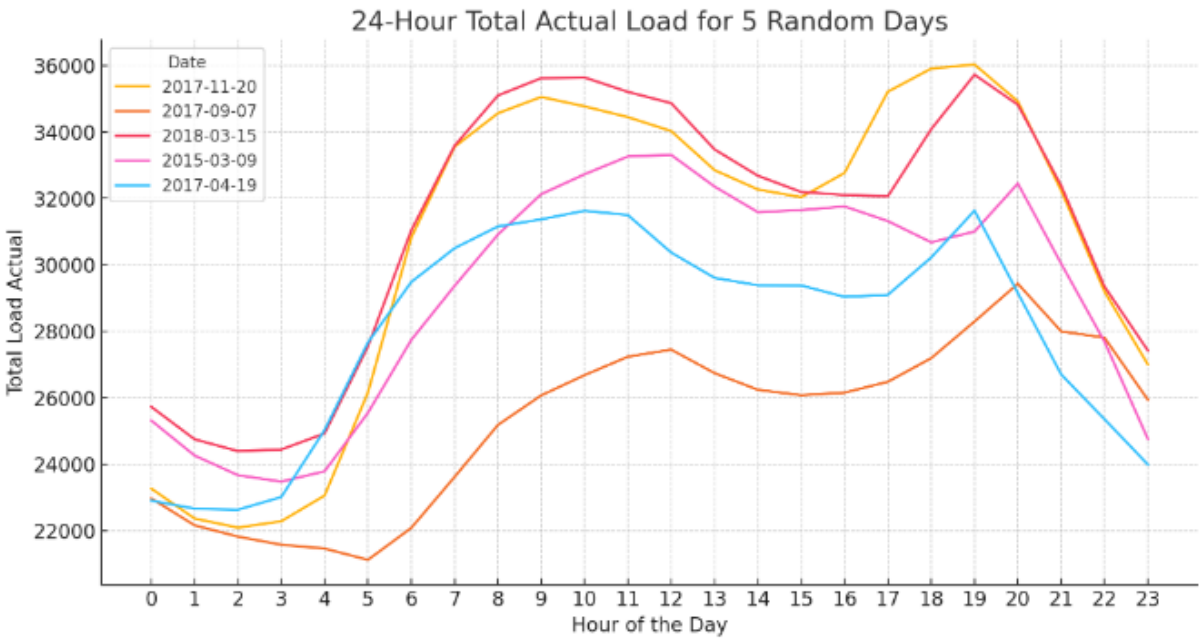

Figure 22: daily seasonality of the actual load.

The plot shows clear patterns in load variations through the day. Peaks (from 9 to 12 in the morning and 19 to 20 in the night) correspond to typical human activity, such as higher loads during working hours and lower loads during the night. This indicates a **daily seasonality**.

- **Weekly Load demand seasonality -daily mean- (7 Days)**:

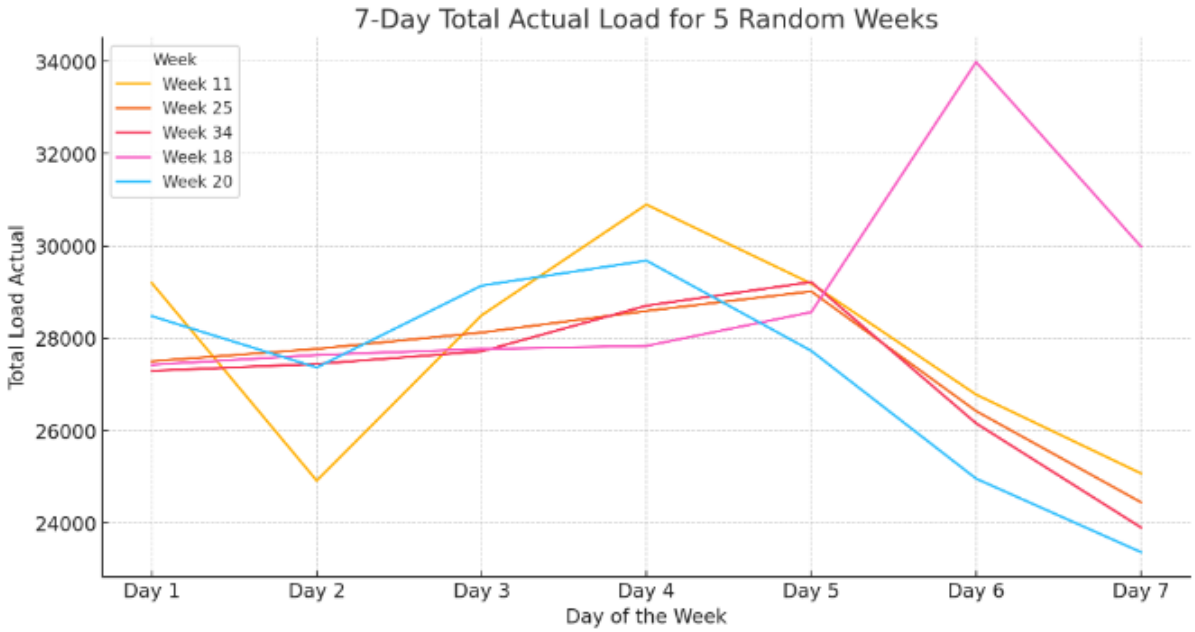

Figure 23: weekly seasonality of the actual load.

Each week's load demand demonstrates consistent variations across the days of the week. High load typically occurs on weekdays, reflecting typical weekly activity patterns and

highlighting weekly seasonality. However, Week 18 exhibits an unusual increase in load during the weekend, which could be attributed to a special event, weather changes, or other external factors influencing demand during that period.

- **Monthly Load -daily mean- (31 Days)**:

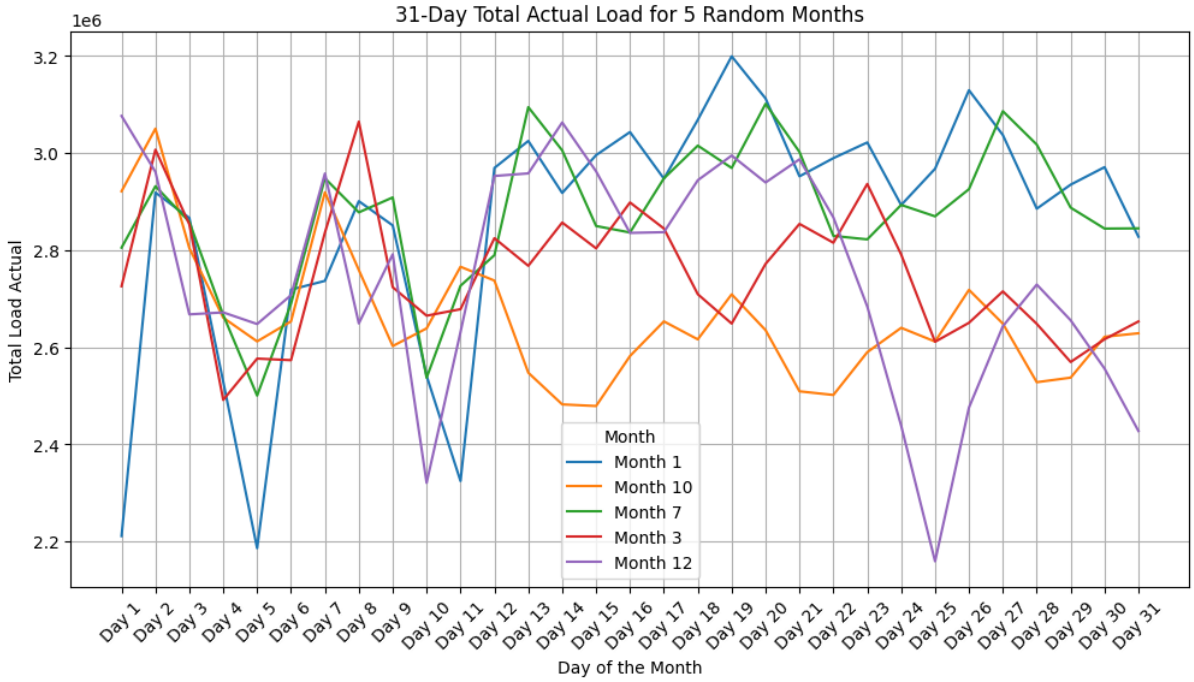

Figure 24: monthly seasonality of the actual load.

The monthly plot shows some interesting trends over 31 days. For the first three weeks, there is a clear pattern: the first two weekdays have spikes in demand, while the weekends drop. But in the third week and after, this pattern changes, and the demand becomes less predictable. This suggests monthly seasonality, with some shifts in behavior as the month progresses, possibly due to factors like holidays, events, or changing routines.

Conclusion:

These findings from the EDA confirm the presence of seasonality in the dataset at daily, weekly, and monthly scales. This insight is crucial for modeling and forecasting, as incorporating seasonality will enhance the model's accuracy.

Bivariate EDA:

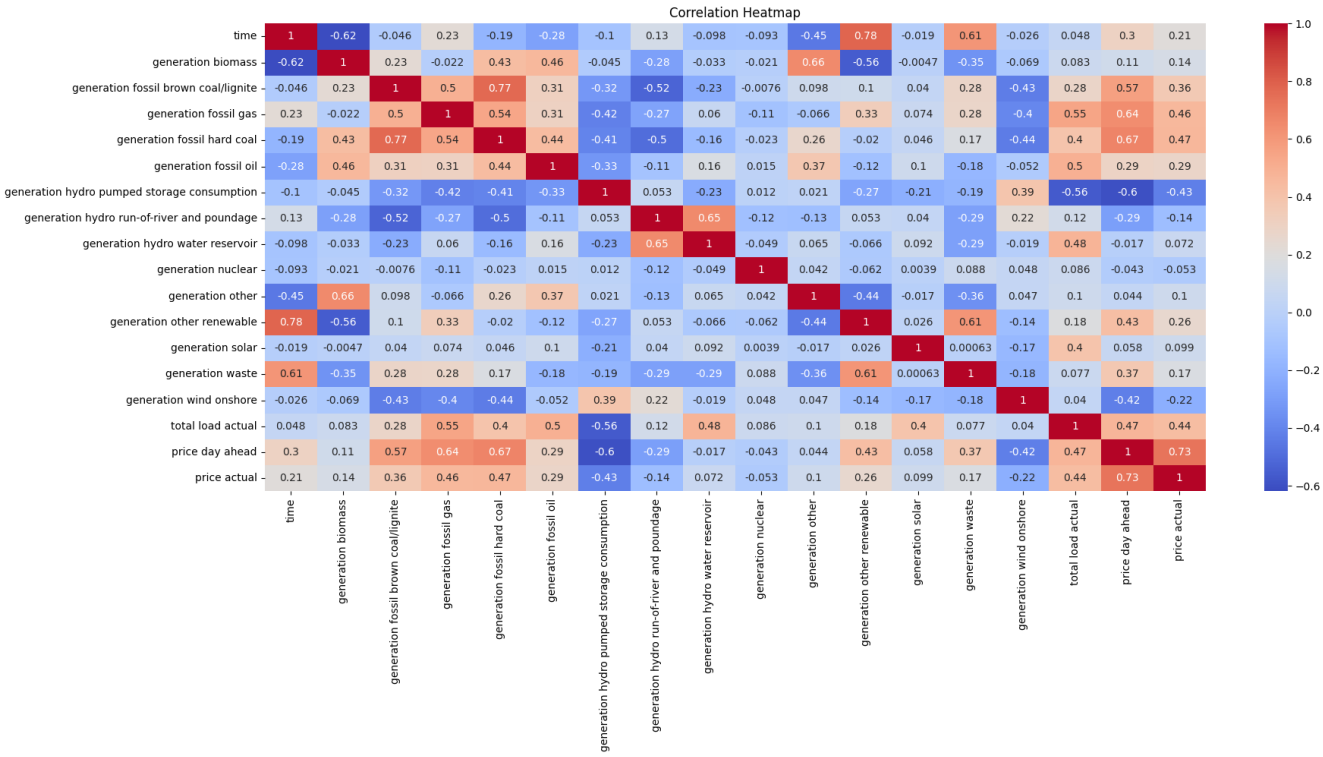

Figure 25: energy_dataset correlation map.

Figure 22 exhibits the correlation coefficients in the form of a correlation matrix, observations of note include:

- **Fossil-based vs. Renewable energy:** Fossil fuels are still strongly correlated with price and load, showing their dominance in the grid. Renewables like wind have a weaker or negative correlation with prices.
- **Load Management:** Load is primarily generated by fossil-based power, with a secondary contribution from renewables.

Producing energy with fossil-based fuels is costly for suppliers. As such, when the price of these fuels fluctuates, the increase in cost results in a proportional increase in costs for customers. Investing in renewable energy is a means to reduce the mass emission of greenhouse gases, while the investment in newer and cleaner fuels has proven to have a great initial cost, the assimilation of which is set to balance the cost over time and is directly correlated with the current load management of non-renewable energy sources.

ML Problem 1(Unsupervised): Feature engineering using dimensionality reduction.

Colab file is in the table page 4.

Dimensionality reduction in time series feature engineering is a crucial process aimed at simplifying complex datasets while preserving essential information. This technique addresses

challenges such as high dimensionality, noise, and redundancy, which can hinder effective analysis and predictive performance. Various methods have been developed to achieve this, each tailored to specific types of time series data and applications. (Zhang, A., & Ma, A. 2021)

The goal is to improve the dataset by reducing similar features (weather features for different cities) into fewer columns, making the price forecasting model more efficient. Since Spain is a small country with similar weather patterns across regions, many weather features for different cities are likely to be highly correlated. By combining these correlated features, we can simplify the dataset without losing important information. This approach follows feature selection techniques, which focus on removing redundant variables to improve machine learning performance (H2O.ai, n.d.).

Despite the advantages, dimensionality reduction can lead to the loss of critical temporal dependencies, potentially impacting model performance. Therefore, careful selection of techniques is essential to balance dimensionality reduction with the preservation of intrinsic data properties ("A Survey on Dimensionality Reduction Techniques for Time-series Data", 2023) (Silva et al., 2022).

The three techniques selected in this report are as follows:

- **Linear data (similar features separated for each city can be simplified):**

  PCA: captures variance in data using orthogonal linear projections (Jolliffe, 2002).

  It simplifies complex datasets, allowing for easier visualization of patterns and relationships.

- **Non-linear data (different weather features are naturally non-linear):**

  t-SNE: is known for its ability to uncover local clusters (Van Der Maaten & Hinton, 2008).

  UMAP: strikes a balance between preserving local and global structure (McInnes et al., 2018).

Execution time comparison:

Technique	Execution time (fitting the dataset) in seconds
PCA	**0.01**
t-SNE	**471.67**
UMAP	**505.51**

Table 1:table of execution time taken to fit twenty features for each technique.

Based on the results achieved on table 1, PCA aligns more with the desired goal. PCA is ideal for speed and variance retention, while UMAP and t-SNE better highlight complex data patterns. To improve the computational compatibility of the dataset, PCA was implemented.

PCA is faster than the other techniques and with variance ratio of 32% for the first component and 16% for the second component as illustrated in Figure 23.

```
PCA variance ratio: [0.32000734 0.16277032]
```

Figure 26: PCA variance ratio.

The first two components explain around 50% of the variance in twenty the features of the weather, meaning that there is some linear relationship between those columns.

Reducing all similar features using PCA:

Now using PCA to reduce the forty-five similar weather conditions columns.

Cumulative explained Variance Ratio:

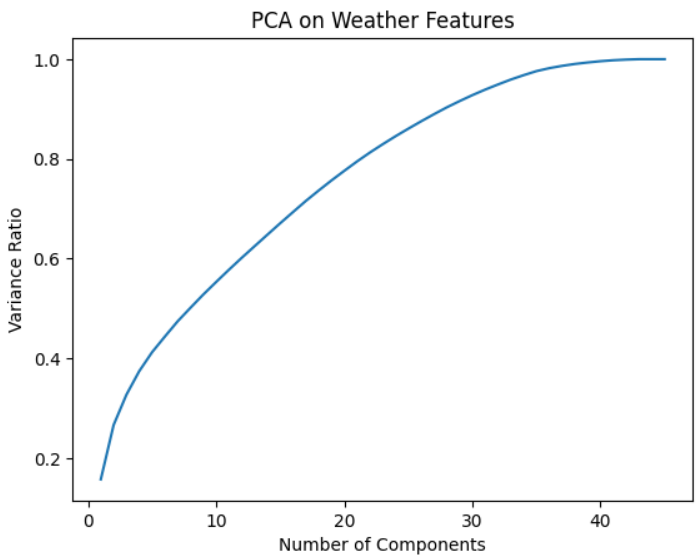

Figure 27: PCA cumulative explained variance ratio.

Number of components	Variance
1	0.15
2	026
3	0.32
4	0.37
5	0.41
6	0.44
7	0.47
8	0.50
9	0.52
10	0.55
11	0.57
12	0.60
13	0.62
14	0.64
15	0.67
16	0.69
17	0.71
18	0.73
19	0.75
20	0.77
21	0.79
22	0.81
23	0.82
24	0.84
25	0.86
26	0.87
27	0.88
28	0.90
29	0.91
30	0.92
31	0.93
32	0.94
33	0.95

Table 2: array of variance of each component.

The cumulative explained variance ratio shows how much variance each principal component accounts for. For example, the first component explains 16% of the variance, and the first thirty-three components explain 95%. Using thirty-three components reduces the dimensionality by 20% of the whole dataset, while retaining 95% of the information, making it a good trade-off for dimensionality reduction.

Creating new dataset with reduced features:

Upon applying dimensionality reduction to the dataset, the number of columns reduced from 63 to 51.

ML Problem 2 (Supervised): Forecasting electricity prices.

Colab file is in the table page 4.

Predicting electricity prices is crucial for energy market stakeholders due to its impact on decision-making, financial planning, and operational efficiency. Accurate forecasting can reduce electricity costs by up to 10% for market participants, helping optimize bidding strategies and manage risks (Weron, 2014). It also supports the integration of renewable energy, which is vital as renewable sources accounted for 29% of global electricity generation in 2020 (IEA, 2021).

Characteristics of our dataset:

Our dataset has patterns of seasonality, with strong autocorrelation meaning that we must use special models that are specialized in this type of forecasting, models that take seasonality and trends in consideration, like:
Prophet, Neural networks, XGBOOST...

Splitting the dataset to 80% training and 20% testing:

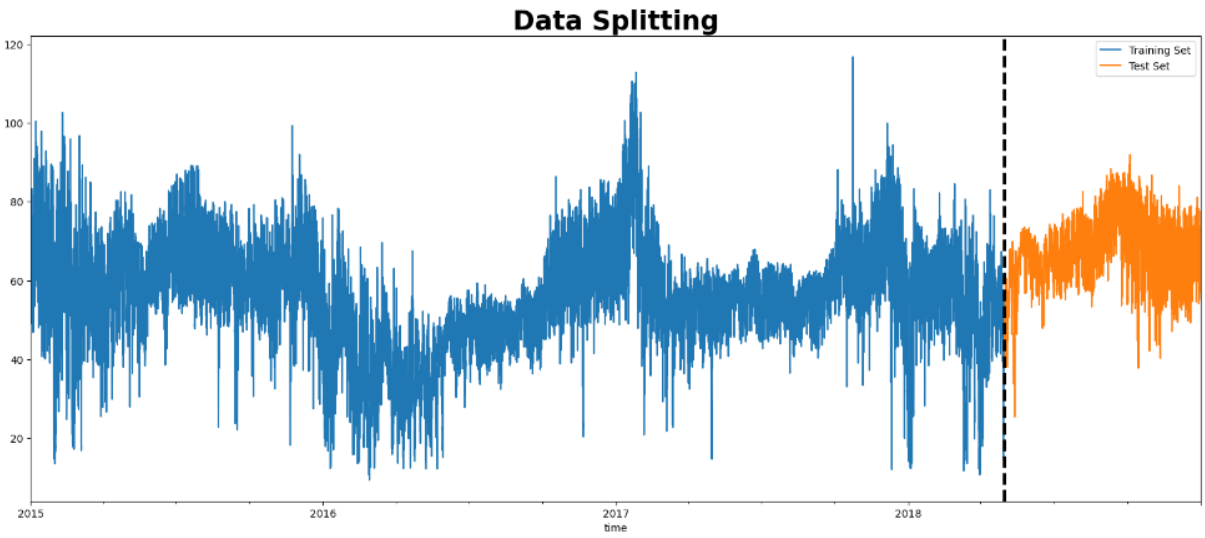

Figure 28: Data splitting.

The selected models:

Prophet: (made by Facebook)

Forecasting electricity prices using the Prophet model has gained traction due to its ability to handle seasonal trends and incorporate external factors. A study on Italian electricity spot prices demonstrated that integrating Prophet with seasonal trend decomposition

techniques reduced the mean absolute percentage error (MAPE) by 18% compared to baseline models (Stefenon et al., 2023). In Kuwait, the Prophet model, when used with multiple regressors, achieved a MAPE of 2.77%, indicating its robustness in forecasting electricity generation (Almazrouee et al., 2020).

The applications that utilize Prophet (aside from evaluating) were inspired from a Kaggle notebook, which is appropriately referenced below (Faaris, 2023).

Benchmarking other models:

I used **PyCaret** to train multiple models, and get the one with the best metrics out of all of them, in our case ***Croston*** came out of on top based on the metrics provided later:

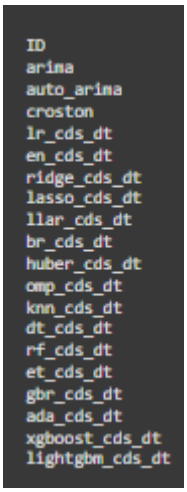

Figure 29: trained models.

Even though it got better metrics, Croson is considered bad to deal with strong seasonality and trends, so even if the model is ranked above XGboost and ARIMA, they would perform better than Croson, but for the sake of examining Croson model, we selected it.

Croston's performs poorly with time series that have strong seasonality. This is because it focuses on estimating the average demand between non-zero observations, ignoring seasonal patterns. (Rizvi, 2024) (Alchakov & Kramar, 2023) (Liu, 2024).

Limitations of Croston's Method:

1. **Designed for Intermittent Data**: It assumes irregular demand and fails to capture regular seasonal fluctuations (Adhikari & Agrawal, 2012).
2. **No Seasonal Modeling**: Croston's method does not account for seasonal effects, leading to poor accuracy when seasonality is significant (Kunst, 2016).

The metrics used to evaluate our models:

Forecasting models are evaluated using various metrics, including Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), Mean Absolute Percentage Error (MAPE), and R-

squared (R^2). Each metric has its strengths and weaknesses, influencing their applicability depending on the data characteristics and error distributions.

MAE: Measures average absolute errors, providing a straightforward interpretation. (Hodson, 2022).

RMSE: Penalizes large errors more due to squaring, making it ideal for measuring error variance (Hodson, 2022; St-Aubin & Agard, 2022).

MAPE: Scale-independent but undefined for zero actual values (Kim & Kim, 2016).

R^2: Measures explained variance, useful for assessing model fit, and more informative than MSE or RMSE (Chicco et al., 2021).

Croston:

Model	MASE	RMSSE	MAE	RMSE	MAPE	SMAPE	TT (Sec)
Croston	0.5357	0.3768	3.2985	3.2985	0.0604	0.0625	0.2633

Figure 30: Croston metrics final dataset.

Croston got good metrics.

Prophet:

These metrics are for the final_dataset before and after dimensionality reduction (reduced_dataset)

	MAE	RMSE	MAPE	R^2
Final_dataset	**5.7938**	**7.7947**	**12.76%**	**0.7**
Reduced_dataset	**5.7938**	**7.7947**	**12.76%**	**0.7**

Table 3: prophet tunning.

While the dimensionality reduction helped reduce the computational compatibility the did not help with improving the metrics.

The prophet metrics are a little bit low, giving an approximate accuracy of 87% (100% - MAPE), but this value could be wrong if the model is affected by outliers.

Tunning our prophet model:

Tunning the model is adding seasonality parameters, and flexibility to adjust to trends.

To improve the Prophet model, I adjusted the **changepoint_prior_scale** to 0.1 to limit sensitivity to minor trend fluctuations, ensuring the model focuses on significant shifts, and set **seasonality_prior_scale** to 10.0 to better capture the pronounced seasonal patterns in electricity price data. These changes slightly improved the metrics, balancing trend detection and seasonal flexibility. However, to achieve optimal parameters, I could have used techniques like **grid search** or **Bayesian optimization**, systematically exploring a range of parameter combinations to maximize the model's performance.

After fitting these are our parameters:

	MAE	RMSE	MAPE	R^2
Final_dataset	**5.7297**	**7.7047**	**12.64%**	**0.7069**
Reduced_dataset	**5.7297**	**7.7047**	**12.64%**	**0.7069**

Table 4: prophet metric after tunning.

Tunning the prophet model managed to get better metrics, with the possibility to get better results through better selection of parameters. However, the reduced dataset still had same metrics again.

Evaluation conclusion:

Dimensionality reduction did not help the prophet model improve overall metrics, while tuning prophet did help, **the Croston model still had better metrics.**

Forecast of the testing split of the dataset:

Croston:

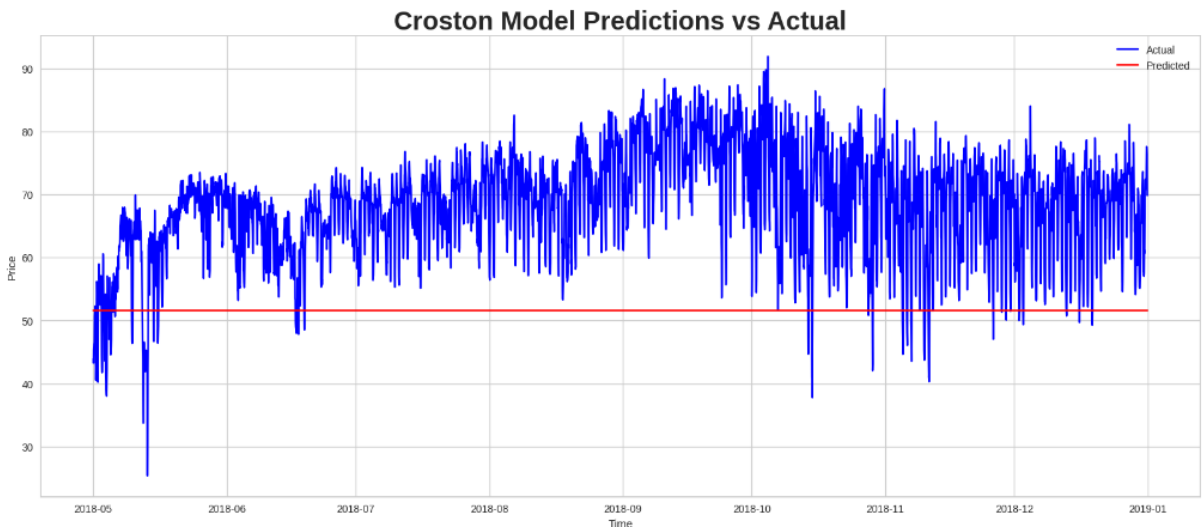

Figure 31: CROSTON MODEL PREDICTION.

Even though Croston had better metrics, the model failed completely to predict the testing value. It does not capture the nuances of rapidly changing seasonal patterns (Yu-qi et al., 2013).

Prophet with the non-feature engineered dataset:

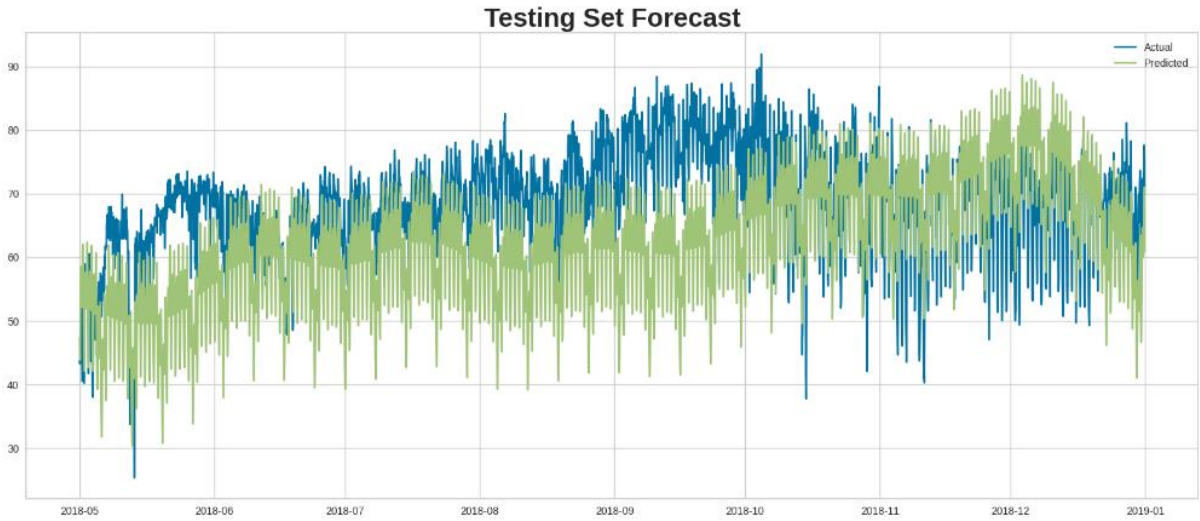

Figure 32: Prophet prediction 1.

Prophet with the feature engineered dataset:

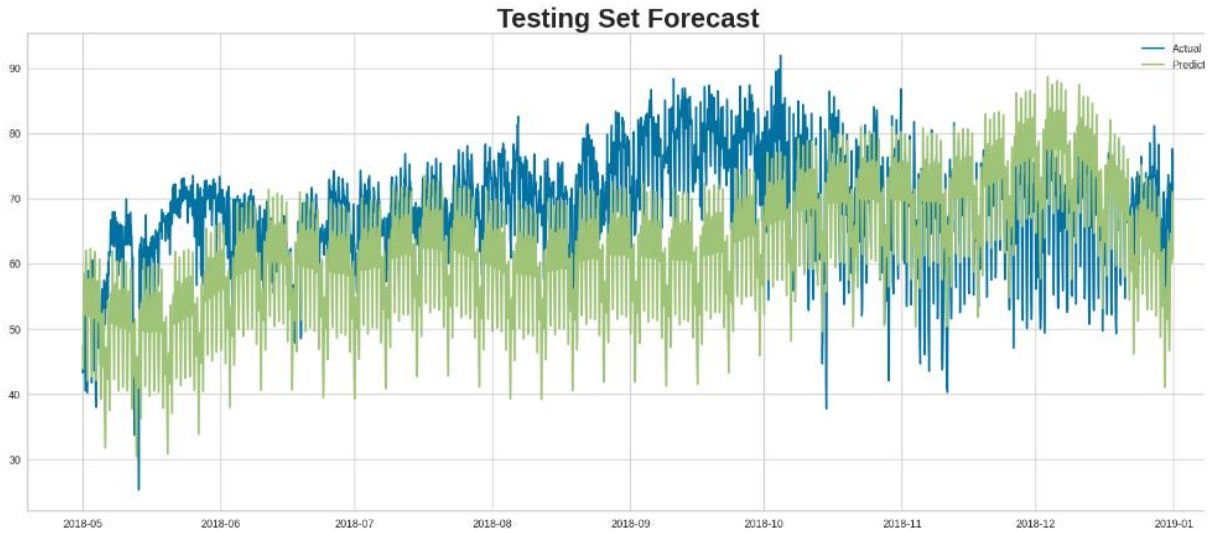

Figure 33: Prophet prediction 2.

Prophet conclusion:

Dimensionality reduction using PCA effectively reduced the dataset from 63 to 51 columns, preserving 95% of the variance and improving computational efficiency. However, it had minimal impact on the Prophet model's predictive performance. While Prophet achieved a MAPE of 12.64% after tuning, Croston's model, despite better metrics, failed to capture seasonal patterns. The study demonstrates that dimensionality reduction aids efficiency but requires careful integration with forecasting models to enhance accuracy.

Predicting future electricity prices (7 days):

Prophet with the non-feature engineered dataset:

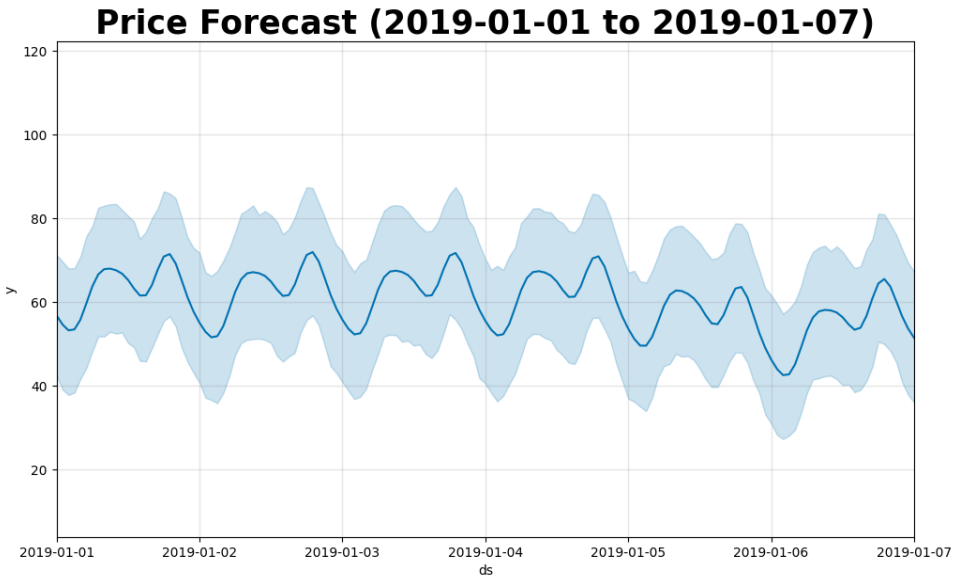

Figure 34: Price forecast by prophet 1.

The prophet in this plot tried to forecast future values of electricity prices, we can clearly see that the model respected the two busy points in each day, showing that prophet does indeed respect seasonality.

Prophet with the feature engineered dataset:

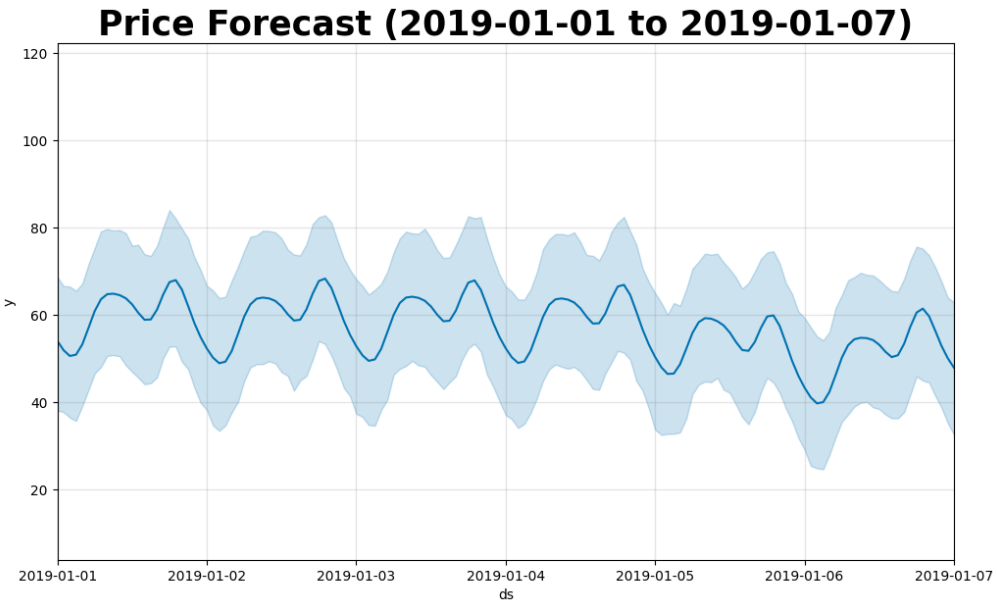

Figure 35: Price forecast by prophet 2.

There is a slight difference in forecasting between the two models, but it is safe to say that there is no major difference in forecasting. However, what we can understand from this is that the metrics that showed everything identical are not very accurate.

Regardless of the accuracy, our model was successful in trying to predict 7 days ahead of the dataset, observing the graph from figure 33, and figure 19, our model seamlessly managed to respect the daily seasonality and trend pattern.

ML Problem 3 (Unsupervised): Clustering based on the correlated feature with the grid load and electricity price.

Colab file is in the table page 4.

Knowing when there are peak hours and when low demand is happening is important for the market participants, and clustering the most important and corelated features, can help with always classifying any datapoint in one of the clusters that represents the state of the demand. This approach is different from forecasting because it clusters right after the actual grid has moved to the demand, means that this is useful to analyse the past data to help the experts analyse the trends and seasonality of the energy demand, which could result in better load management, efficiency of energy generation that would result in lowering the price...etc.

The primary features that are used in this clustering are the actual price, and the actual load necessarily, as they stand for the demand and offer directly. The other feature that is well correlated with the primary features according to our correlation heat-map is:

- **generation fossil gas** strongly correlates well with both the price and the load (Is the source of load generating the biggest amount of power), making him the prefect feature for this application.

K-means clustering is a method used to group data into different clusters based on similarity. It works by assigning each data point to the closest group centre and then adjusting the group centres to reduce the differences within each group. The process repeats until the groups are stable. This method is useful for handling large datasets and is used in areas like image processing and data mining.

- **Image Denoising**: K-means is used to group image data for noise reduction, enhancing image quality through adaptive techniques (Agrawal et al., 2015).
- **Data Retrieval**: The algorithm organizes data for efficient retrieval, making it valuable in spatial data mining (Sharma et al., 2012).

Normalizing features using a scaler is essential for enhancing the performance of clustering algorithms. Normalization adjusts the scale of features, allowing them to contribute equally to distance metrics like Euclidean distance, which is sensitive to feature magnitude (Souto et al., 2008). Techniques like modified Robust Scaling can effectively handle outliers, improving clustering accuracy by reducing their influence on the clustering process (Prasad & Srikanth,

2024).

Methods and metrics used to get the best k-number:

The Elbow method and average silhouette score serve as essential techniques for determining the optimal number of clusters.

Elbow method:

This method helps to select the best k-number, as we can see the cross of the two plots is between 6 and 5, so we need to use a metric to make sure that we will select the best k-number.

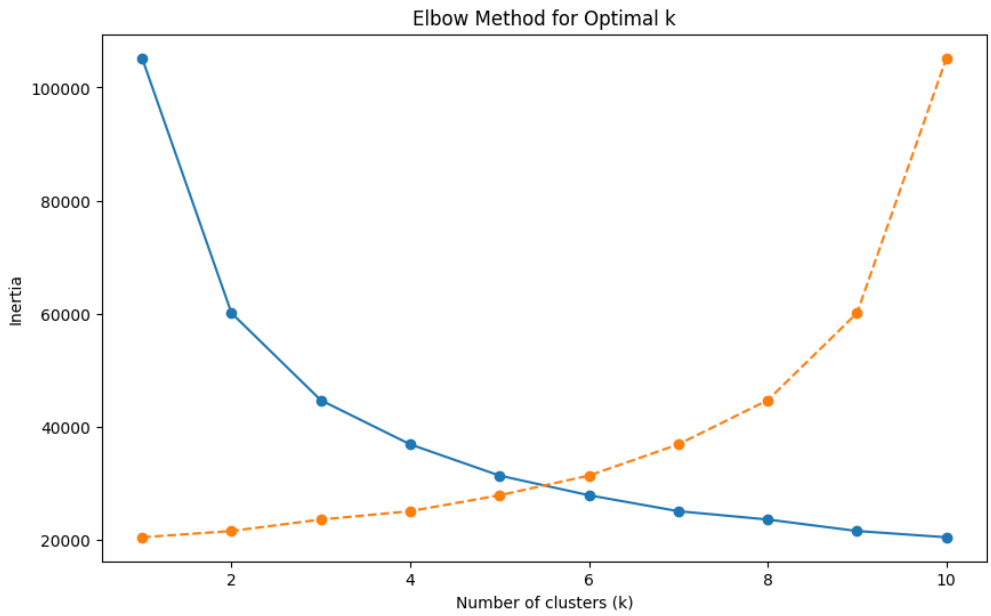

Figure 36: elbow method for the specified features.

average silhouette score:

The average silhouette score for optimal clusters in various studies often corroborated the findings from the Elbow method, enhancing the reliability of the clustering results(Rahmawati et al., 2024).

This table shows the findings:

Number of clusters	Average silhouette score
4	0.2863
5	0.2888
6	0.2812

Table 5: Average silhouette score.

The most optimal k-number is 5 with a fair average silhouette score of 0.2888.

The results of clustering using k-means:

Raw results:

Based on the clustering results, we can interpret what each cluster represents:

Cluster	Fossil Gas Generation (MW)	Total Load (MW)	Average Price (€/MWh)	Description
0	3913.7	24257.09	37.37	Represents off-peak hours with lower demand and cheaper energy sources. The relatively low fossil gas generation and low total load lead to the lowest electricity price.
1	5857.8	31599.11	71.25	Groups peak demand periods when expensive gas generation is not heavily needed to meet high consumption. This cluster corresponds to a moderate level of fossil gas generation and high total load.
2	5040.9	31468.2	51.64	Represents transitional periods between different clusters or market states. The moderate fossil gas generation and high total load lead to a moderate electricity price.
3	4716.2	24575.5	58.8	Indicates periods of supply constraints or unexpected demand spikes. The relatively low fossil gas generation and low total load result in a relatively high price.
4	10096.9	33925.7	69.2	Represents peak demand periods requiring maximum fossil gas generation. This cluster corresponds to the highest fossil gas generation and the highest total load, leading to a high price.

Table 6: Raw clustering results.

Plotting the results:

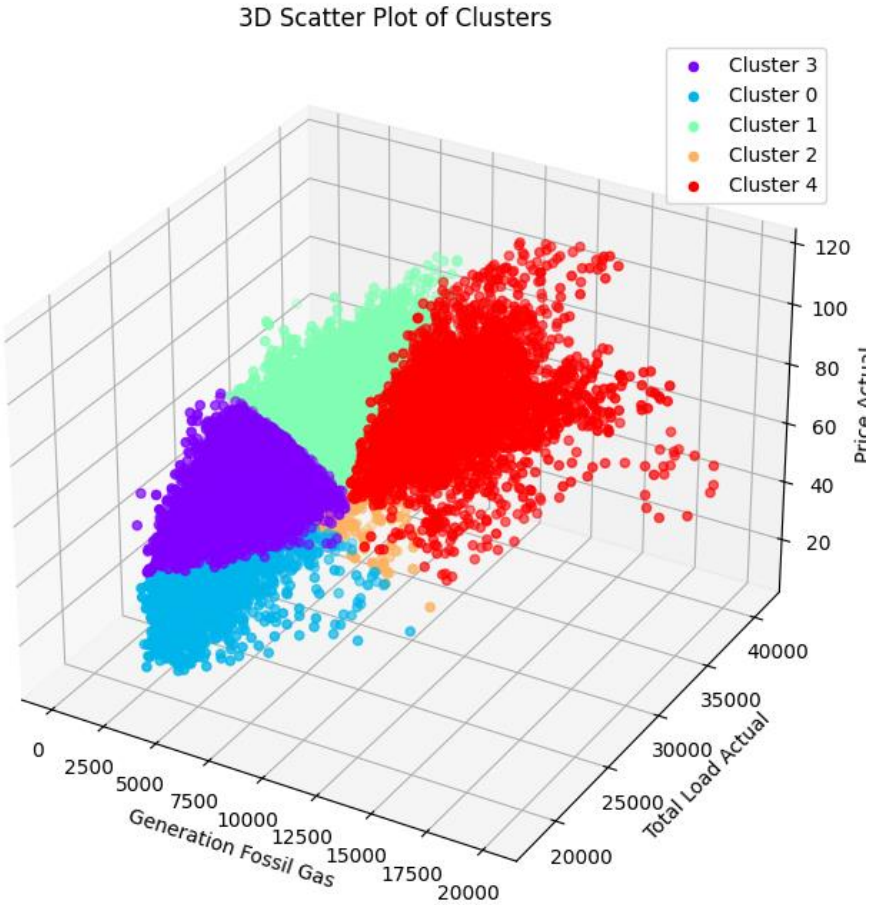

Figure 37: 3D plotting of clusters.

After plotting, we can easily see the relationship between each cluster, we can see cluster 2 being stuck between all other clusters representing the transitional period between the not neighbouring clusters, like cluster 0 (all low off-peak), and cluster 4 (all high peak). We also see clusters that are neighbours and close to each other statistically.

For example: clusters 4 and 1 are peak demand with different fossil gas generation.

These clusters help identify patterns in energy generation, demand, and pricing, which can be useful for energy planning and market analysis.

Conclusion:

This report shows the importance of data preparation, exploratory data analysis (EDA), and machine learning techniques in predicting electricity prices. Throughout the study, we faced challenges in both unsupervised and supervised learning.

Preprocessing and EDA played a key role in overcoming these challenges. Cleaning the data and organizing it helped remove errors and reduce noise. EDA also helped understand the trends,

seasonality, and outliers in the data, guiding in selecting the right features for the models. This initial work was crucial for training and testing the models, showing how important it is to properly prepare data for accurate predictions.

For the first unsupervised problem, we used dimensionality reduction techniques like PCA, t-SNE, and UMAP to reduce redundancy in the weather-related features. PCA was the best method, keeping 95% of the data's important information while reducing its size by 20%. t-SNE and UMAP, were slower and less effective for large-scale tasks.

In the supervised problem, we faced challenges with models like Croston and Prophet. Croston did not handle the seasonal patterns in electricity prices well, making it unsuitable. Prophet was better at modeling trends and seasonality, but it needed tunning to perform better. Although dimensionality reduction helped speed up the training, it did not improve the accuracy of the forecasts, showing that choosing the right features is more important.

For the clustering task, we used K-means to group data based on factors like electricity price, total load, and fossil gas generation. By normalizing the data and using methods like the Elbow method, we found five distinct clusters that show different demand periods. These clusters help understand peak and off-peak times, which can help energy market participants manage generation and pricing better.

In conclusion, this study shows how effective data preparation, EDA, and machine learning can be in solving real-world problems like predicting electricity prices. While we made good progress, future work should focus on improving models, selecting the best features, and adding external factors like renewable energy and market trends to make predictions even more accurate and support better energy management.

References:

Almazrouee, A.I., Almeshal, A.M., Almutairi, A.S., Alenezi, M.R., Alhajeri, S.N. and Alshammari, F.M. (2020). Forecasting of Electrical Generation Using Prophet and Multiple Seasonality of Holt–Winters Models: A Case Study of Kuwait. *Applied Sciences*, 10(23), p.8412. doi:https://doi.org/10.3390/app10238412.

Anon, (n.d.).

Weron, R. (2018). Recent advances in electricity price forecasting: A review of probabilistic forecasting. Renewable and Sustainable Energy Reviews, 81, 1548–1568. https://doi.org/10.1016/j.rser.2017.05.234

Chatgpt.com. (2024). *ChatGPT*. [online] Available at: https://chatgpt.com/c/678f54f8-5710-8008-b0c9-805722439f47 [Accessed 21 Jan. 2025].

Chicco, D., Warrens, M.J. and Jurman, G. (2021). The Coefficient of Determination R-squared Is More Informative than SMAPE, MAE, MAPE, MSE and RMSE in Regression Analysis Evaluation. *PeerJ Computer Science*, [online] 7(5), p.e623. doi:https://doi.org/10.7717/peerj-cs.623.

de, P., Daniel, Costa, I.G., Rodrigo, Ludermir, T.B. and Schliep, A. (2008). Comparative study on normalization procedures for cluster analysis of gene expression datasets. *CiteSeer X (The Pennsylvania State University)*, [online] pp.2792–2798. doi:https://doi.org/10.1109/ijcnn.2008.4634191.

Dettling, M. (2018). *Applied Time Series Analysis SS 2018*. [online] Available at: https://ethz.ch/content/dam/ethz/special-interest/math/statistics/sfs/Education/Advanced%20Studies%20in%20Applied%20Statistics/course-material-1719/Zeitreihen/ATSA_Script.pdf?utm_source=chatgpt.com [Accessed 22 Jan. 2025].

dimitriosroussis (2021). *Electricity price forecasting with DNNs (+ EDA)*. [online] Kaggle.com. Available at: https://www.kaggle.com/code/dimitriosroussis/electricity-price-forecasting-with-dnns-eda [Accessed 22 Jan. 2025].

Facebook (2019). *Prophet*. [online] Prophet. Available at: https://facebook.github.io/prophet/.

Hodson, T.O. (2022a). Root-mean-square error (RMSE) or mean absolute error (MAE): when to use them or not. *Geoscientific Model Development*, [online] 15(14), pp.5481–5487. Available at: https://gmd.copernicus.org/articles/15/5481/2022/.

Hodson, T.O. (2022b). Root-mean-square error (RMSE) or mean absolute error (MAE): when to use them or not. *Geoscientific Model Development*, [online] 15(14), pp.5481–5487. Available at: https://gmd.copernicus.org/articles/15/5481/2022/.

Hodson, T.O. (2022c). Root-mean-square error (RMSE) or mean absolute error (MAE): when to use them or not. *Geoscientific Model Development*, [online] 15(14), pp.5481–5487. Available at: https://gmd.copernicus.org/articles/15/5481/2022/.

Ieee.org. (2024). *Seasonal time series forecasting with a state-dependent model*. [online] Available at: https://ieeexplore.ieee.org/document/6640818 [Accessed 22 Jan. 2025].

Irfan, U. (2024). *Even solar energy's biggest fans are underestimating it*. [online] Vox. Available at: https://www.vox.com/climate/372852/solar-power-energy-growth-record-us-climate-china?utm_source=chatgpt.com.

Ratnadip, Adhikari., Ramesh, Kumar, Agrawal. (2012). 1. Forecasting strong seasonal time series with artificial neural networks.

Weron, R. (2014). Electricity price forecasting: A review of the state-of-the-art with a look into the future. *International Journal of Forecasting*, 30(4), 1030–1081. https://doi.org/10.1016/j.ijforecast.2014.08.008Nowotarski, J., &

КИРИЛО ЄМЕЦЬ (2024). МЕТОДИ ПРОГНОЗУВАННЯ ЧАСОВИХ РЯДІВ З ВИРАЖЕНОЮ СЕЗОННІСТЮ НА ОСНОВІ ТРАНСФОРМЕРІВ. *Herald of Khmelnytskyi National University Technical sciences*, [online] 333(2), pp.131–134. doi:https://doi.org/10.31891/2307-5732-2024-333-2-20.

Le, L.-H. (2024). Time series analysis and applications in data analysis, forecasting and prediction. *HPU2 Journal of Science Natural Sciences and Technology*, 3(1), pp.20–29. doi:https://doi.org/10.56764/hpu2.jos.2024.3.1.20-29.

Liu, C. (2024). Time-series water prediction based on feature clustering fusion and XGboost. *Applied and Computational Engineering*, [online] 95(1), pp.86–94. doi:https://doi.org/10.54254/2755-2721/95/2024bj0062.

mfaaris (2022). *3 Ways to Deal with Time Series Forecasting*. [online] Kaggle.com. Available at: https://www.kaggle.com/code/mfaaris/3-ways-to-deal-with-time-series-forecasting [Accessed 22 Jan. 2025].

Mohd Faizan Rizvi (2024). ARIMA Model Time Series Forecasting. *International Journal for Research in Applied Science and Engineering Technology*, 12(5), pp.3782–3785. doi:https://doi.org/10.22214/ijraset.2024.62416.

Nasser, M., Salim, N., Saeed, F., Basurra, S., Rabiu, I., Hamza, H. and Alsoufi, M.A. (2022). Feature Reduction for Molecular Similarity Searching Based on Autoencoder Deep Learning. *Biomolecules*, [online] 12(4), p.508. doi:https://doi.org/10.3390/biom12040508.

Prasad, S. (2024). Time Series. *Advanced Statistical Methods*, pp.109–162. doi:https://doi.org/10.1007/978-981-99-7257-9_3.

Preprints.org. (2024). *Enhancing K-Means Clustering Accuracy through Modified Robust Scaling Technique - [v1]*. [online] Available at: https://www.preprints.org/manuscript/202411.1245/v1 [Accessed 8 Jan. 2025].

ResearchGate. (2021). *Seasonality in High Frequency Time Series*. [online] Available at: https://www.researchgate.net/publication/350068336_Seasonality_in_High_Frequency_Time_Series?utm_source=chatgpt.com [Accessed 22 Jan. 2025].

Sheo Yon Jhin, Lee, J., Jo, M., Kook, S., Jeon, J., Jihyeon Hyeong, Kim, J. and Park, N. (2022). EXIT: Extrapolation and Interpolation-based Neural Controlled Differential Equations for Time-series Classification and Forecasting. *Proceedings of the ACM Web Conference 2022*. doi:https://doi.org/10.1145/3485447.3512030.

the (2014). *What is dimensionality reduction? What is the difference between feature selection and extraction?* [online] Data Science Stack Exchange. Available at: https://datascience.stackexchange.com/questions/130/what-is-dimensionality-reduction-what-is-the-difference-between-feature-selecti?utm_source=chatgpt.com [Accessed 21 Jan. 2025].

umap-learn.readthedocs.io. (n.d.). *UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction — umap 0.5 documentation*. [online] Available at: https://umap-learn.readthedocs.io/en/latest/.

В.В. Альчаков and В.А. Крамарь (2019). ОЦЕНКА МЕТОДОВ МАШИННОГО ОБУЧЕНИЯ ДЛЯ ПРОГНОЗИРОВАНИЯ СЕЗОННЫХ ВРЕМЕННЫХ РЯДОВ. *Известия ЮФУ. Технические науки.*, [online] (2). Available at: https://izv-tn.tti.sfedu.ru/index.php/izv_tn/article/view/790 [Accessed 21 Jan. 2025].

www.ikigailabs.io. (n.d.). *How to Choose the Best Model for Time Series Forecasting: ARIMA, Prophet, or mSSa*. [online] Available at: https://www.ikigailabs.io/blog/how-to-choose-the-best-model-for-time-series-forecasting-arima-prophet-or-mssa.

Zhang, S. (2024). The Atlantic. *The Atlantic*. [online] doi:https://doi.org/10x1519/1080x1080.